

Design and Evaluation of a Segmented Enterprise Security Architecture Using JWS Authentication, FortiGate Security Controls, and Wireshark Traffic Analysis

Suhaib Kashif

Department of Computer and Software Engineering
Information Technology University, Lahore, Pakistan
Email: suhaibkashif03@gmail.com

Abstract—Modern enterprise web applications require layered protection across application, transport, and network layers. Token-based authentication mechanisms such as JSON Web Signature (JWS) provide integrity protection for authentication claims, but they do not provide confidentiality when transmitted over plaintext HTTP. Similarly, network-layer availability attacks such as SYN flooding can disrupt access to otherwise correctly authenticated services. This paper presents the design, implementation, and evaluation of a segmented enterprise-style cybersecurity architecture deployed in EVE-NG. The environment consists of a DMZ-hosted Node.js web application using JWS authentication, an internal SOC client, a Kali Linux red-team host, an edge router, and a FortiGate firewall providing segmentation, intrusion prevention, and anomaly-based SYN flood detection. Wireshark was used to inspect authentication and authorization flows under HTTP and compare them against HTTPS/TLS-protected traffic. The project further evaluates token tampering, unauthorized access attempts, replay attack exposure, and SYN flood detection. Results show that JWS effectively detects token tampering but does not protect token confidentiality over HTTP, while HTTPS prevents passive token interception. FortiGate anomaly logs successfully detected `tcp_syn_flood` activity originating from the red-team network and applied session-clearing actions. The work demonstrates a complete security lifecycle: secure design, attack simulation, packet-level analysis, detection, and mitigation.

Index Terms—JWS, JWT, FortiGate, Wireshark, EVE-NG, HTTP, HTTPS, TLS, Replay Attack, SYN Flood, DMZ, SOC, IPS, Network Segmentation

I. INTRODUCTION

Enterprise web applications are exposed to multiple classes of security threats, including credential interception, token replay, unauthorized access, token manipulation, and availability attacks. A secure system must therefore not rely on a single control. Instead, it should combine application-layer authentication, encrypted communication, network segmentation, firewall enforcement, traffic monitoring, and attack detection.

This project began as an implementation of HTTP authentication using JSON Web Signature (JWS), but was expanded into a complete enterprise-style security architecture. The final environment was built in EVE-NG and incorporated a DMZ web server, an internal SOC client, a Kali Linux attacker machine, an edge router, and a FortiGate firewall. The purpose

was to demonstrate not only how JWS authentication works, but also how it behaves under attack scenarios and how additional controls improve the overall security posture.

The project addresses three major security properties:

- **Integrity:** JWS signing ensures that modified tokens are detected.
- **Confidentiality:** HTTPS/TLS prevents credentials and tokens from being exposed in transit.
- **Availability:** FortiGate anomaly detection and DoS controls identify SYN flood behavior.

The paper is written in a research-style format to document the architecture, implementation, experimental evaluation, findings, and recommendations.

II. MOTIVATION

Many student security projects demonstrate isolated tools or simple scripts, but real-world security engineering requires integration of multiple layers. A secure application must be evaluated not only at the code level, but also in terms of how it is deployed inside the network.

This project was designed with the following motivation:

- To build a practical security project suitable for academic and professional showcasing.
- To demonstrate the difference between authentication, authorization, integrity, and confidentiality.
- To show how an attacker can abuse valid tokens if they are exposed over HTTP.
- To demonstrate FortiGate-based detection and mitigation of a SYN flood attack.
- To provide packet-level evidence using Wireshark rather than only relying on application output.

III. BACKGROUND

A. JSON Web Signature

JSON Web Signature is a mechanism used to digitally sign JSON-based data. In token-based authentication systems, a signed token allows the server to verify that the token contents have not been modified. A JWS token generally follows the structure:

```
BASE64URL(Header) .BASE64URL(Payload) .BASE64URL(
Signature)
```

The header defines the signing algorithm and token type. The payload contains claims such as username, role, issued-at time, and expiration time. The signature is computed using a secret or private key. If the token is modified, the signature verification fails.

B. JWS vs Confidentiality

JWS provides integrity and authenticity, but it does not encrypt the payload. Anyone who captures the token can decode its header and payload. Therefore, JWS should be transmitted over HTTPS in production systems. Without HTTPS, a bearer token may be captured and replayed by an attacker before expiration.

C. Replay Attack

A replay attack occurs when an attacker captures valid authentication material and reuses it to gain access. In bearer-token systems, possession of the token is often enough to access protected resources until the token expires or is revoked. This makes token confidentiality extremely important.

D. SYN Flood Attack

A SYN flood is a denial-of-service attack where an attacker sends a large number of TCP SYN packets to a target system. The goal is to exhaust server or network resources by forcing the target to handle excessive half-open TCP connection attempts. Firewalls and intrusion prevention systems can detect SYN flood patterns and respond by dropping or clearing suspicious sessions.

IV. PROJECT OBJECTIVES

The objectives of this project were:

- Design a segmented enterprise network in EVE-NG.
- Deploy a DMZ-hosted web application using Node.js and Express.js.
- Implement JWS-based authentication and authorization.
- Analyze HTTP authentication traffic using Wireshark.
- Demonstrate unauthorized access prevention.
- Demonstrate token tampering detection through JWS signature validation.
- Analyze replay attack risk when tokens are transmitted over HTTP.
- Implement HTTPS to protect credentials and tokens in transit.
- Simulate SYN flood traffic from Kali Linux.
- Configure FortiGate firewall policies, IPS filtering, and anomaly detection.
- Validate detection using FortiGate logs and Wireshark captures.

V. SYSTEM ARCHITECTURE

A. Topology Overview

The lab was deployed in EVE-NG using a segmented topology. The FortiGate firewall was placed between the edge router and three security zones: DMZ, Internal LAN, and Red Team network.

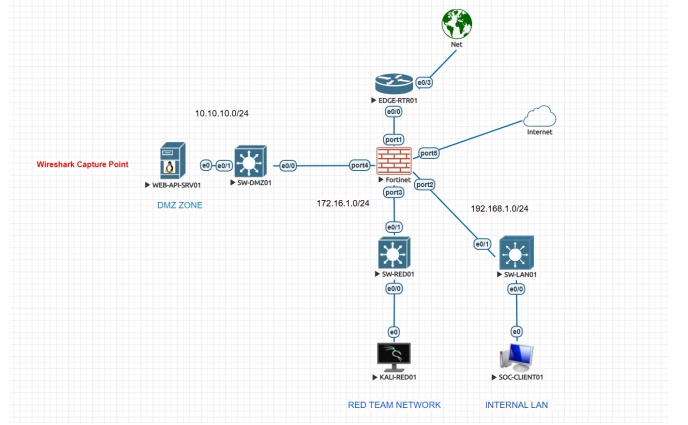


Fig. 1: Final segmented enterprise security topology deployed in EVE-NG.

The architecture contains the following main components:

- **EDGE-RTR01:** Edge router connected to the upstream NAT network.
- **FortiGate Firewall:** Enforces inter-zone policies and detects attacks.
- **WEB-API-SRV01:** Ubuntu-based DMZ server hosting the JWS web application.
- **SOC-CLIENT01:** Internal client used for legitimate access and monitoring.
- **KALI-RED01:** Attacker host used for replay testing and SYN flooding.
- **Cloud0/Management Network:** Provides management access to FortiGate.

B. Network Segmentation

The topology was divided into separate zones to follow enterprise security design principles.

TABLE I: Security Zones

Zone	Subnet	Purpose
DMZ	10.10.10.0/24	Web application hosting
Internal LAN	192.168.1.0/24	SOC client and legitimate access
Red Team	172.16.1.0/24	Attack simulation
WAN/Transit	10.10.20.0/24	Router-to-FortiGate transit
Management	192.168.149.0/24	Firewall management

C. IP Addressing

VI. THREAT MODEL

The project assumes an attacker located in the Red Team network. The attacker can send traffic to the DMZ web

TABLE II: IP Addressing Plan

Device/Interface	Role	IP Address
FortiGate port1	WAN/Transit	10.10.20.2/24
FortiGate port2	Internal LAN	192.168.1.1/24
FortiGate port3	Red Team	172.16.1.1/24
FortiGate port4	DMZ	10.10.10.1/24
FortiGate port5	Management	192.168.149.250/24
WEB-API-SRV01	DMZ Server	10.10.10.10/24
SOC-CLIENT01	Internal Client	192.168.1.10/24
KALI-RED01	Red Team Host	172.16.1.10/24

application for testing purposes but should not be allowed to access the Internal LAN. The attacker attempts the following:

- Access protected resources without authentication.
- Modify a valid JWS token to bypass authorization.
- Capture and reuse a valid token transmitted over HTTP.
- Disrupt service availability using SYN flooding.

The defender controls the application, FortiGate firewall, and SOC monitoring workflow. The defender's objectives are:

- Enforce authentication and authorization.
- Detect token tampering.
- Reduce token exposure using HTTPS.
- Segment the network to limit lateral movement.
- Detect and respond to SYN flood behavior.

VII. APPLICATION DESIGN AND IMPLEMENTATION

A. Web Application

The web application was developed using Node.js and Express.js. It provided both a browser-based SOC dashboard and REST API endpoints for authentication and protected data access.

TABLE III: Application Endpoints

Endpoint	Method	Purpose
/	GET	SOC dashboard
/login	POST	Authenticate user and issue JWS token
/secure-data	GET	Protected resource requiring valid token

B. JWS Token Generation

During login, the server validates credentials and generates a signed token containing the username and role. The token is then returned to the client.

Listing 1: Login request used to generate a JWS token

```
curl -X POST http://10.10.10.10:3000/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin123"}'
```

A valid response contains a token beginning with:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

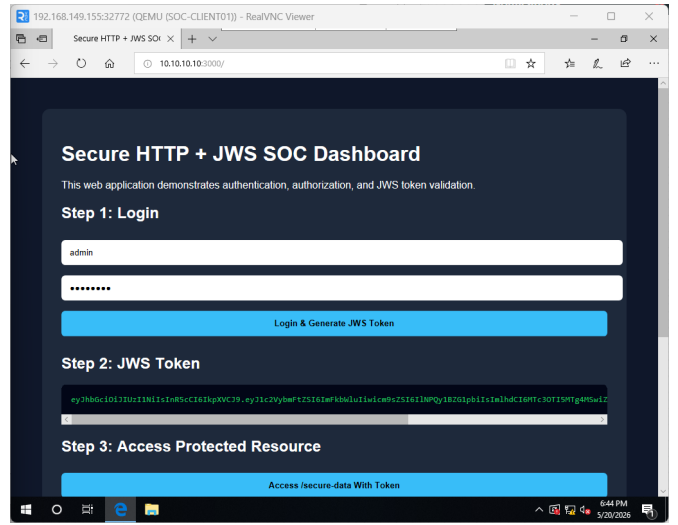


Fig. 2: SOC client accessing the web authentication dashboard.

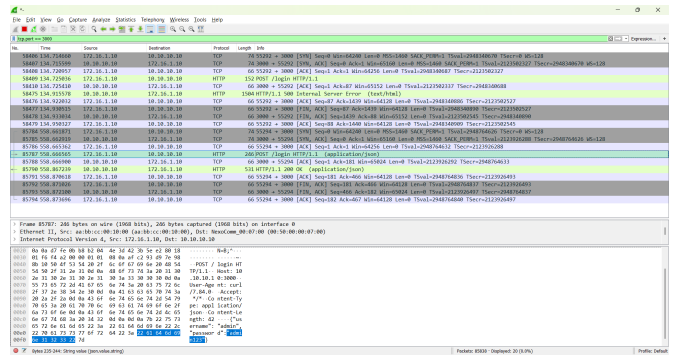


Fig. 3: Wireshark capture showing HTTP POST login request.

VIII. AUTHENTICATION AND AUTHORIZATION ANALYSIS

A. Authentication over HTTP

Wireshark was used to capture the login request. Since the initial application used HTTP, the username and password were visible in plaintext.

This confirms that plaintext HTTP exposes sensitive authentication data to network observers.

B. Authorization using Bearer Token

After login, the client accessed the protected endpoint by sending the JWS token in the Authorization header.

Listing 2: Protected resource access with JWS token

```
curl http://10.10.10.10:3000/secure-data \
-H "Authorization: Bearer VALID_JWS_TOKEN"
```

This screenshot is significant because it shows that the bearer token can be observed over HTTP. This creates replay attack exposure.

IX. UNAUTHORIZED ACCESS TESTING

The protected endpoint was accessed without a token.

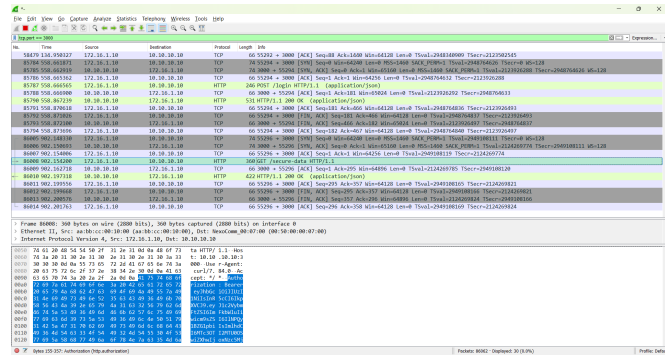


Fig. 4: Wireshark capture showing JWS token inside HTTP Authorization header.

Listing 3: Unauthorized request without token

```
curl http://10.10.10.10:3000/secure-data
```

Expected response:

```
{
  "message": "Token Missing"
}
```

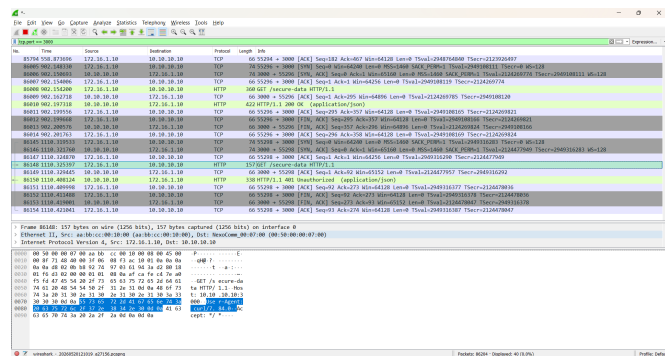


Fig. 5: Unauthorized access attempt rejected due to missing token.

This test verifies that the application enforces authorization before allowing access to protected data.

X. TOKEN TAMPERING EVALUATION

A token tampering test was conducted by modifying one or more characters in a valid token and resending it to the protected endpoint.

Listing 4: Tampered token request

```
curl http://10.10.10.10:3000/secure-data \
-H "Authorization: Bearer MODIFIED_TOKEN"
```

Expected response:

```
{
  "message": "Invalid or Tampered Token"
}
```

This demonstrates the core value of JWS: token integrity validation. If the token is modified, the server detects the mismatch and denies access.

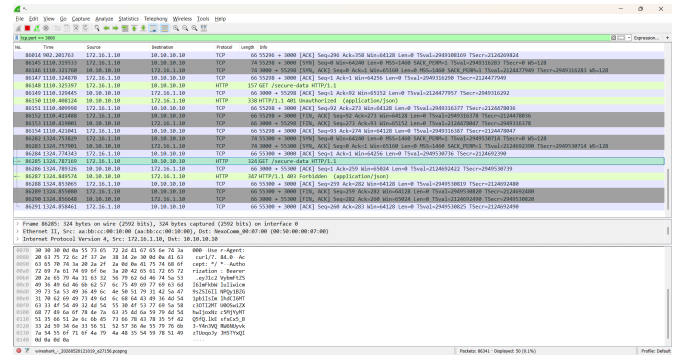


Fig. 6: Modified JWS token rejected after signature verification failure.

XI. REPLAY ATTACK ANALYSIS

A. Attack Rationale

A replay attack was considered because the JWS token was transmitted over HTTP. Since JWS does not encrypt token contents or protect confidentiality, a captured bearer token can be reused by an attacker while it remains valid.

No separate Kali screenshot is required for this section because the primary evidence is the Wireshark authorization capture shown in Fig. 4. That capture proves that the bearer token is visible in transit over HTTP. An attacker who can observe that traffic can copy the token and reuse it.

B. Replay Attack Procedure

The replay attack procedure is summarized below:

- 1) SOC client logs in and obtains a valid JWS token.
- 2) Wireshark captures the Authorization header over HTTP.
- 3) The attacker extracts the bearer token.
- 4) The attacker sends the same token in a new request.
- 5) The server accepts the request if the token is valid and unexpired.

Listing 5: Replay attack command format

```
curl http://10.10.10.10:3000/secure-data \
-H "Authorization: Bearer CAPTURED_VALID_TOKEN"
```

C. Security Interpretation

This attack does not break JWS cryptography. Instead, it abuses bearer token semantics. A valid bearer token grants access to whoever possesses it. Therefore, token confidentiality is essential.

The replay test produced the following conclusion:

- JWS prevents token modification.
- JWS does not prevent token theft.
- HTTP exposes bearer tokens.
- HTTPS is required to protect tokens in transit.

XII. SYN FLOOD ATTACK SIMULATION

A. Attack Setup

A SYN flood attack was launched from the Kali Linux host in the Red Team network against the DMZ web server. The attack targeted TCP port 3000, where the HTTP version of the Node.js application was hosted.

Listing 6: SYN flood attack from Kali Linux

```
sudo hping3 -S --flood -p 3000 10.10.10.10
```

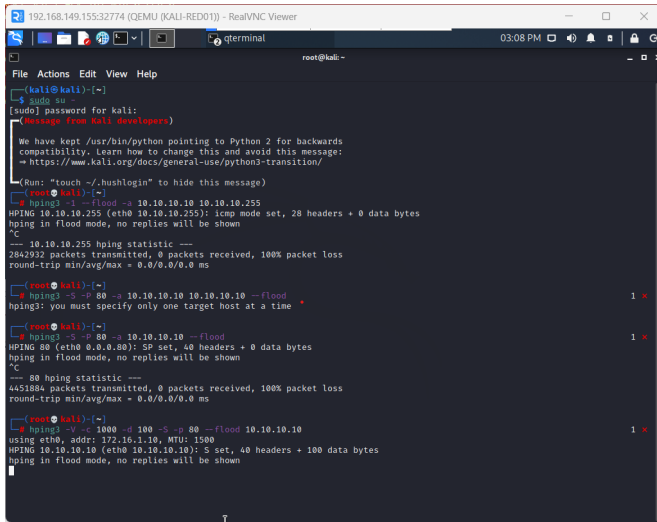


Fig. 7: SYN flood attack launched from Kali Linux.

B. Packet-Level Observation

Wireshark was used to observe repeated SYN packets toward the web server.

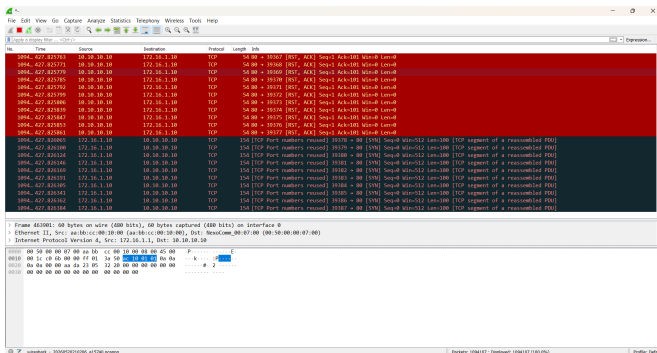


Fig. 8: Wireshark capture showing SYN flood traffic targeting the DMZ web server.

This test demonstrates an availability threat against the application infrastructure.

XIII. FORTIGATE SECURITY CONTROLS

A. Interface Segmentation

FortiGate was configured with dedicated interfaces for WAN, LAN, Red Team, DMZ, and management access.

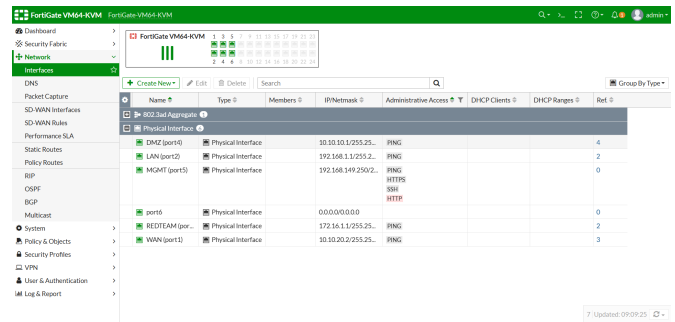


Fig. 9: FortiGate interface configuration for segmented zones.

B. Firewall Policy Enforcement

Firewall policies were configured to allow required communication while restricting unnecessary access. The SOC client was allowed to access the DMZ web server. The Red Team network was allowed to access the DMZ for controlled testing. No policy was created to allow Red Team access to the Internal LAN.

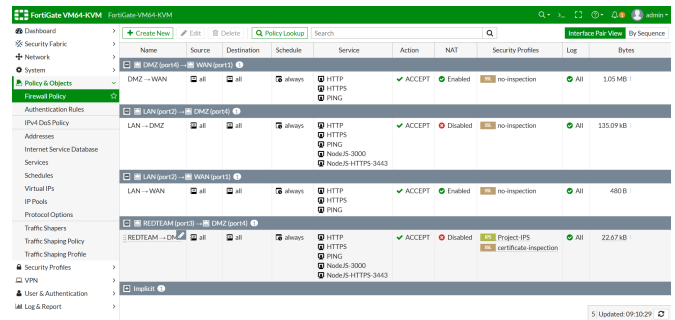


Fig. 10: FortiGate firewall policies enforcing inter-zone access control.

The policy model follows the principle of least privilege. Red Team traffic is restricted to the DMZ and does not have direct access to internal SOC resources.

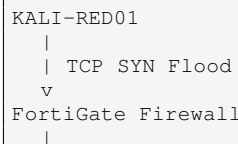
C. Intrusion Prevention System

An IPS profile named Project-IPS was created and attached to the Red Team to DMZ policy. It focused on server-side Linux HTTP/TCP signatures with medium, high, and critical severity. Packet logging was enabled.

XIV. SYN FLOOD DETECTION AND MITIGATION

FortiGate anomaly logs successfully detected the attack as tcp_syn_flood. The source IP was 172.16.1.10, corresponding to KALI-RED01. The action was clear_session, indicating that FortiGate actively responded to the anomalous traffic.

The detection workflow can be summarized as:



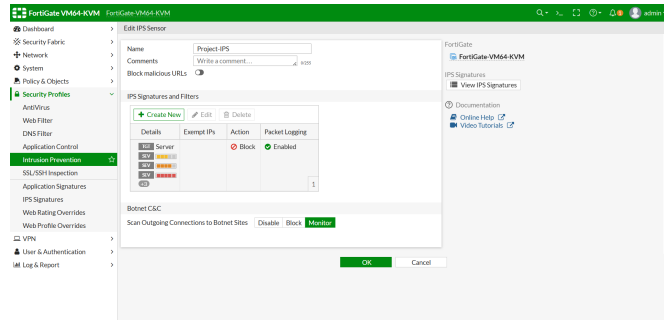


Fig. 11: FortiGate IPS profile configured for Linux HTTP/TCP server protection.

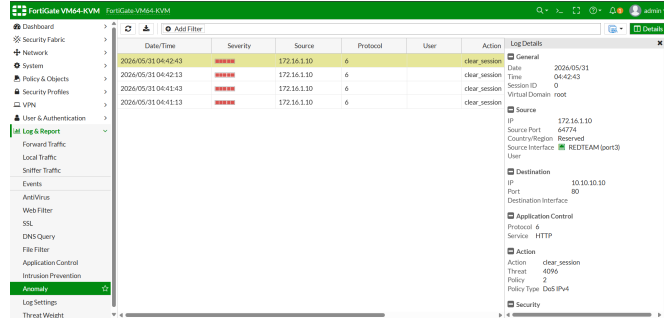


Fig. 12: FortiGate anomaly log showing tcp_syn_flood detection and response.

```
| Detects tcp_syn_flood
| Action: clear_session
v
DMZ Web Server
```

This validates the attack-detection-response lifecycle in the lab.

XV. HTTPS IMPLEMENTATION

A. Reason for HTTPS

The HTTP analysis showed that credentials and bearer tokens were visible in Wireshark. To mitigate this exposure, HTTPS was implemented on port 3443 using a self-signed certificate.

Listing 7: Self-signed certificate generation

```
openssl req -x509 -newkey rsa:4096 \
-keyout key.pem \
-out cert.pem \
-days 365 \
-nodes
```

The application was configured to support both:

- HTTP on port 3000 for comparison and attack demonstration.
- HTTPS on port 3443 for secure communication.

B. TLS Traffic Analysis

After HTTPS was enabled, Wireshark no longer displayed the username, password, token, or Authorization header. In-

stead, it showed TLS handshake messages and encrypted application data.

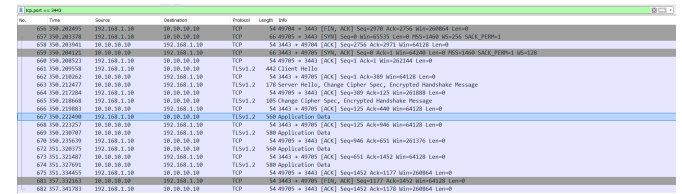


Fig. 13: Wireshark capture showing TLS handshake and encrypted application data.

This demonstrates that HTTPS mitigates passive token interception and reduces replay attack exposure from network sniffing.

XVI. EXPERIMENTAL EVALUATION

A. Test Cases

TABLE IV: Security Test Cases

Test Case	Expected Result	Observed Result
Valid login	Token issued	Token generated
Protected access with token	Access granted	Access granted
Access without token	Denied	Denied
Tampered token	Denied	Denied
HTTP token capture	Token visible	Token visible
HTTPS token capture	Token hidden	Encrypted traffic
SYN flood	Detected by firewall	tcp_syn_flood logged

B. Security Properties Evaluated

TABLE V: Security Properties Demonstrated

Security Property	Mechanism	Evidence
Authentication	Login endpoint	Token generated
Authorization	Bearer token	Secure data access
Integrity	JWS signature	Tampered token rejected
Confidentiality	HTTPS/TLS	Token hidden in Wireshark
Availability	FortiGate DoS detection	tcp_syn_flood log
Segmentation	Firewall policies	Zone-based access control

XVII. RESULTS AND DISCUSSION

The results confirm that JWS is effective for verifying token integrity. When a token was modified, the server rejected it. However, the results also show that JWS alone is insufficient for secure deployment because tokens remain visible over HTTP. The replay attack analysis highlights the risk of transmitting bearer tokens without encryption.

HTTPS successfully addressed this issue by encrypting the entire application payload. Wireshark could observe TLS handshakes and encrypted application data, but it could not reveal credentials or bearer tokens.

At the network level, FortiGate successfully enforced segmentation and detected SYN flood behavior. The anomaly log identifying `tcp_syn_flood` from 172.16.1.10 provides strong evidence of detection and response. This shows how application-layer authentication and network-layer protection can be combined to build a more realistic security architecture.

XVIII. SECURITY RECOMMENDATIONS

Based on the implementation and testing, the following recommendations are proposed:

- Enforce HTTPS for all authentication and token-based communication.
- Disable plaintext HTTP in production environments.
- Use short-lived access tokens.
- Implement refresh-token rotation.
- Add token revocation or blacklist functionality.
- Store signing secrets in environment variables or a secure secret manager.
- Apply rate limiting to login and protected endpoints.
- Use FortiGate IPS and anomaly detection for public-facing services.
- Restrict attacker or untrusted networks from accessing internal LAN resources.
- Enable centralized logging and alerting for SOC visibility.

XIX. LIMITATIONS

This project was implemented in a controlled EVE-NG environment. The SYN flood was generated from a single Kali Linux machine, so it represents a DoS simulation rather than a distributed DDoS attack. The HTTPS certificate was self-signed for lab purposes. In a production deployment, a trusted certificate authority should be used. The application used a static demo user and local signing secret; production environments should integrate identity management, secure password storage, secret rotation, and centralized logging.

The replay attack was analyzed using HTTP token visibility in Wireshark rather than a dedicated screenshot of Kali reusing the token. This is acceptable because the key security evidence is the visibility of the bearer token in plaintext traffic. The HTTPS comparison demonstrates the mitigation.

XX. FUTURE WORK

Future enhancements may include:

- Deploying a SIEM such as Wazuh or Splunk for centralized log monitoring.
- Adding NGINX as a reverse proxy with production-grade TLS configuration.
- Implementing token revocation and refresh-token rotation.
- Adding MFA to strengthen user authentication.
- Implementing rate limiting middleware in Node.js.
- Adding alerting for FortiGate anomaly events.
- Creating incident response playbooks for replay and DoS attacks.
- Extending the architecture with Active Directory and role-based access control.

XXI. CONCLUSION

This project designed and evaluated a segmented enterprise security architecture using JWS authentication, FortiGate firewall controls, Wireshark traffic analysis, attack simulation, and HTTPS protection. The implementation demonstrated secure authentication, authorization, token validation, token tampering detection, replay attack exposure, SYN flood detection, and firewall-based response. The results show that JWS provides strong integrity protection but must be combined with HTTPS to protect confidentiality. FortiGate provided effective segmentation and detected SYN flood activity from the red-team network. Overall, the project demonstrates a complete cybersecurity workflow from system design to attack simulation, detection, mitigation, and security analysis, making it suitable for professional portfolio, GitHub, LinkedIn, CV, and master's application use.

REFERENCES

- [1] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Signature (JWS)," RFC 7515, May 2015.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015.
- [3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018.
- [4] Fortinet, "FortiGate Administration Guide," Fortinet Documentation Library.
- [5] Wireshark Foundation, "Wireshark User's Guide."
- [6] S. Sanfilippo, "hping3: A command-line oriented TCP/IP packet assembler/analyzer," hping project documentation.